Construct tree

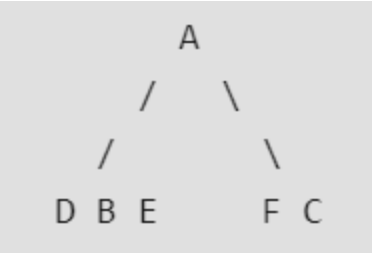
Construct Tree from given Inorder and Preorder traversals

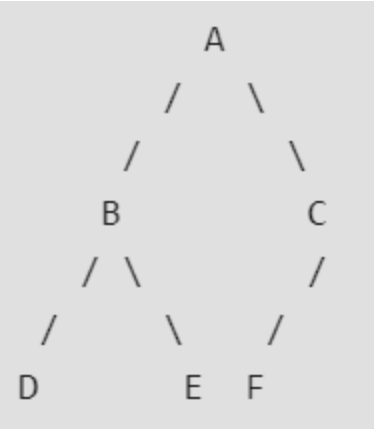
```
preorder = [3,9,20,15,7]
inorder = [9,3,15,20,7]
```

```
    3
   / \
  9   20
     /  \
    15   7
```

Inorder sequence: D B E A F C

Preorder sequence: A B D E C F





- Example

InOrder(root) visits nodes in the following order:

4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:

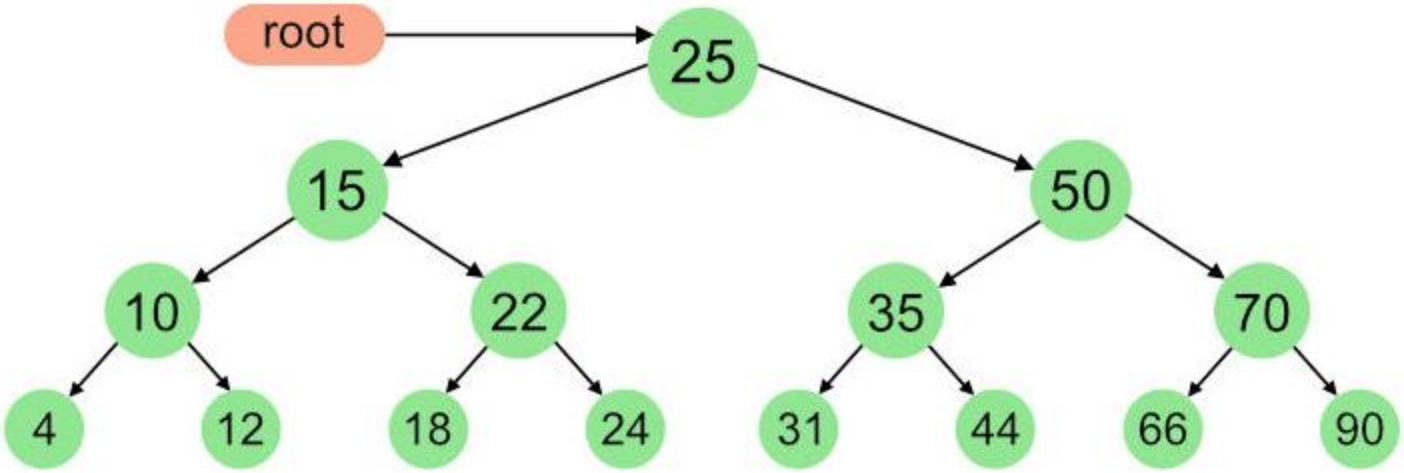
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

InOrder(root) visits nodes in the following order:

4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:

25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90



buildTree()

1) Pick an element from Preorder. Increment a Preorder Index Variable (preIndex in below code) to pick next element in next recursive call.
2) Create a new tree node tNode with the data as picked element.
3) Find the picked element's index in Inorder. Let the index be inIndex.
4) Call buildTree for elements before inIndex and make the built tree as left subtree of tNode.
5) Call buildTree for elements after inIndex and make the built tree as right subtree of tNode.
6) return tNode.

Construct a Binary Tree from Postorder and Inorder

```
in[]   = {2, 1, 3}
post[] = {2, 3, 1}

Output: Root of below tree
      1
    /   \
   2     3
```

```
Input:
in[]   = {4, 8, 2, 5, 1, 6, 3, 7}
post[] = {8, 4, 5, 2, 6, 7, 3, 1}

Output: Root of below tree
          1
       /     \
     2         3
   /    \   /   \
  4     5   6    7
    \
      8
```

Process of constructing tree from in[] = {4, 8, 2, 5, 1, 6, 3, 7} and post[] = {8, 4, 5, 2, 6, 7, 3, 1}

1) We first find the last node in post[]. The last node is "1", we know this value is root as root always appear in the end of postorder traversal.
2) We search "1" in in[] to find left and right subtrees of root. Everything on left of "1" in in[] is in left subtree and everything on right is in right subtree.

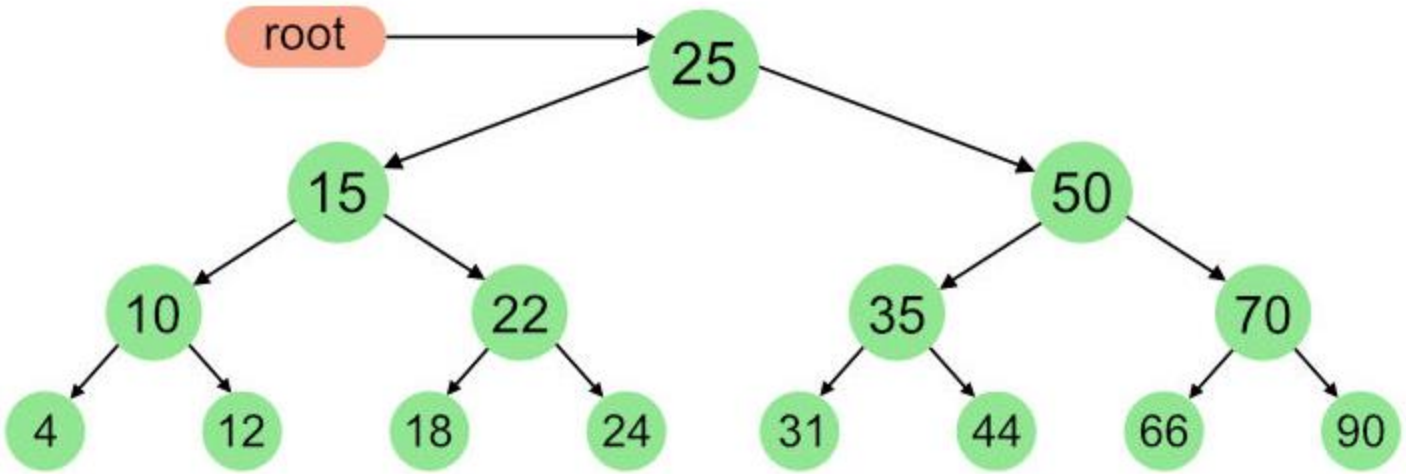
3) We recur the above process for following two.
….**b)** Recur for in[] = {6, 3, 7} and post[] = {6, 7, 3}
…….Make the created tree as right child of root.
….**a)** Recur for in[] = {4, 8, 2, 5} and post[] = {8, 4, 5, 2}.
…….Make the created tree as left child of root.

InOrder(root) visits nodes in the following order:

4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



Full Binary Tree

```
        1
      /   \
    2       3
  /  \     /  \
 4    5   6    7
```

```
      1
    /   \
  2       3
        /   \
      4       5
            /   \
          6     7
```

```
          1
        /   \
      2       3
    /  \     /  \
   4    5   6    7
 /  \
8    9
```

- It is not possible to construct a general Binary Tree from preorder and postorder traversals. But if know that the Binary Tree is Full, we can construct the tree without ambiguity.

Construct Full Binary Tree from given preorder and postorder traversals

pre[] = {1, 2, 4, 8, 9, 5, 3, 6, 7}

post[] = {8, 9, 4, 5, 2, 6, 7, 3, 1}

- 1 is root and 2 is left child
- So 2 is root of all nodes in left subtree
- Ultimately all nodes before 2 in post[] must be in left subtree
- {8, 9, 4, 5, 2} are in left subtree, and the elements {6, 7, 3} are in right subtree

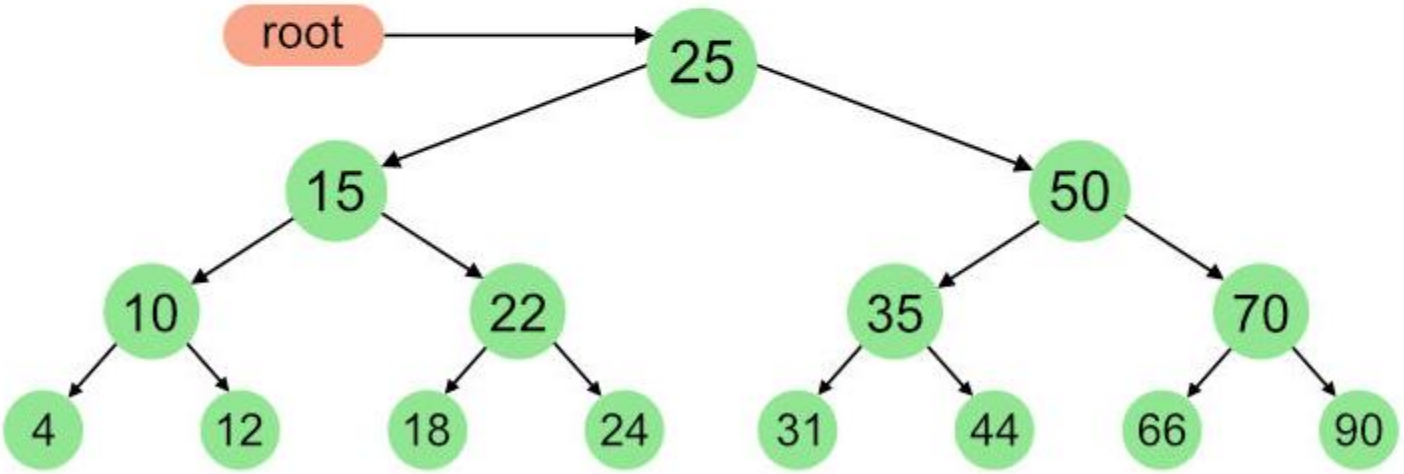
```
              1
            /   \
           /     \
  {8, 9, 4, 5, 2}     {6, 7, 3}
```

```
            1
          /   \
         /     \
{8, 9, 4, 5, 2}    {6, 7, 3}
```

```
          1
        /   \
      2       3
    /  \     /  \
   4    5   6    7
  / \
 8   9
```

A Pre-order traversal visits nodes in the following order:

25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:

4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25